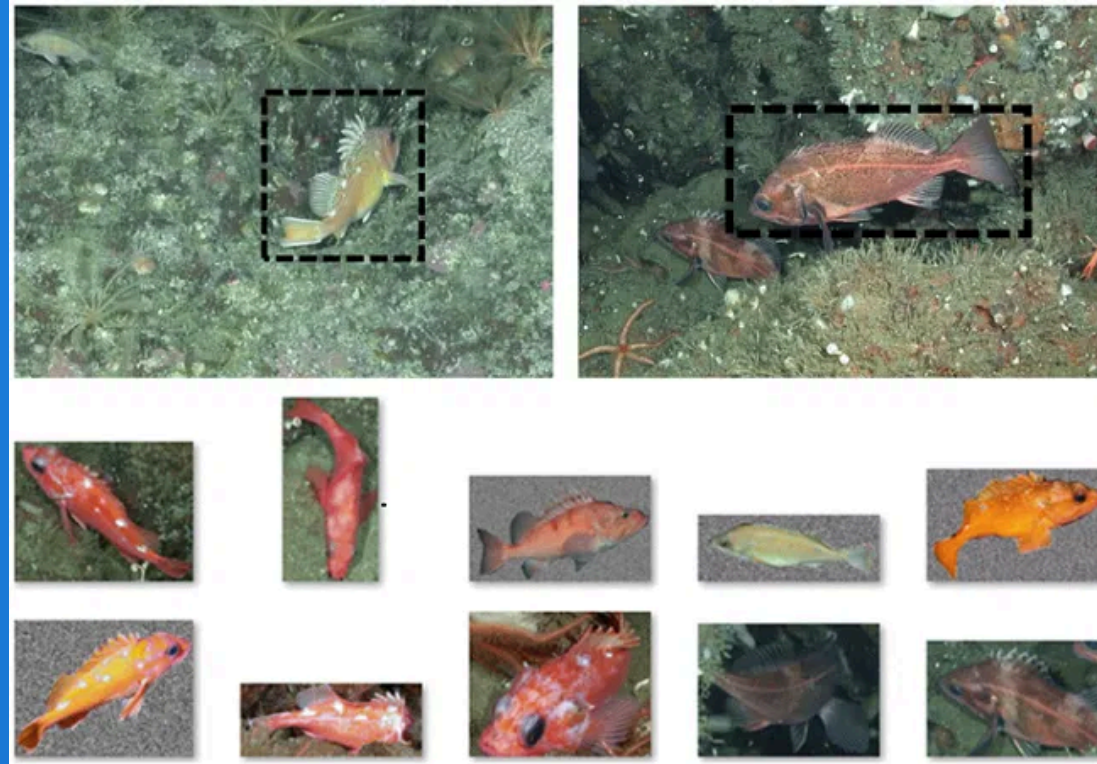


Multiclass Fish Image Classification

Deep Learning Project



Skills Gained: Deep Learning, Python, TensorFlow/Keras, Streamlit, Data Preprocessing, Transfer Learning, Model Evaluation

Dataset Overview

Dataset Structure

The dataset consists of fish images organized into folders by species, making it ideal for supervised learning.

Loading Method

Images are loaded using **TensorFlow's ImageDataGenerator** for efficient batch processing and augmentation.

Key Characteristics

- Multiple fish species categories
- Varied backgrounds and lighting conditions
- Different angles and orientations
- Various sizes and resolutions

Preprocessing Requirements

Images need resizing to uniform dimensions and normalization to [0,1] range before model training.



Data Preprocessing & Augmentation

Image Preprocessing

 **Resizing:** Standardize all images to same dimensions (e.g., 224×224)

 **Normalization:** Rescale pixel values to [0,1] range

Data Augmentation Techniques

 **Rotation:** Random rotations ($\pm 20^\circ$)

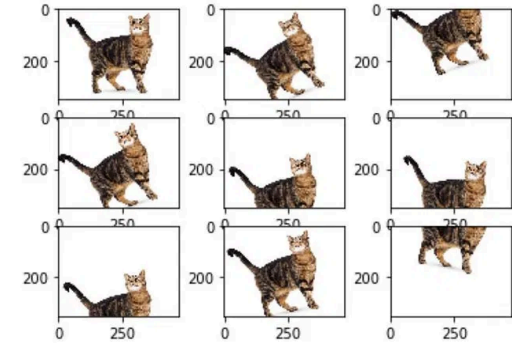
 **Zoom:** Random zoom ($\pm 20\%$)

 **Flip:** Horizontal flipping

 **Brightness/Contrast:** Slight variations

Benefits

Increases effective dataset size and improves model robustness against variations in real-world images



CNN Architecture

What is a CNN?

Convolutional Neural Networks are specialized deep learning models designed for processing structured grid data like images.

Key Layers

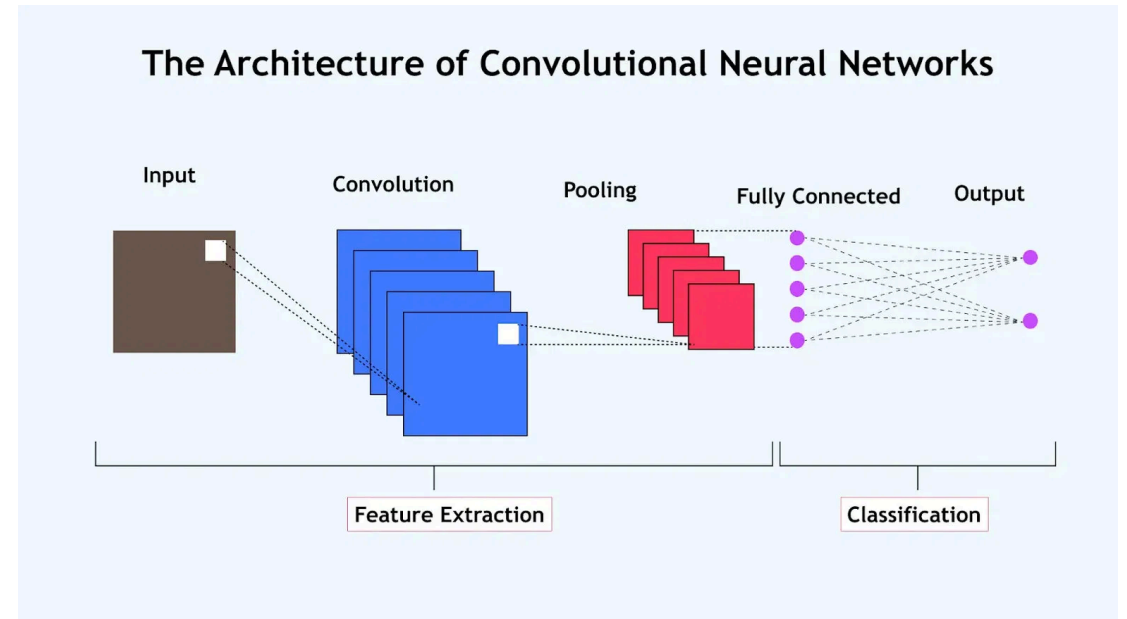
- **Convolutional Layers:** Extract features using filters
- **Pooling Layers:** Reduce spatial dimensions
- **Fully Connected Layers:** Perform classification

For Fish Classification

Our CNN model will learn to identify distinctive features of different fish species such as fin shapes, color patterns, and body structures.

Implementation

We'll build a CNN from scratch using TensorFlow/Keras with multiple convolutional and pooling layers followed by dense layers for classification.



Transfer Learning

What is Transfer Learning?

A technique that reuses a pre-trained model as a starting point for a new task, leveraging knowledge gained from solving one problem to improve generalization in another.

Benefits for Fish Classification

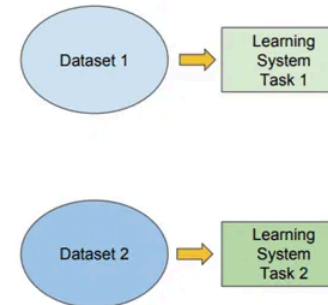
- Reduces training time significantly
- Requires less training data
- Improves model performance
- Prevents overfitting on small datasets

Pre-trained Models Used

- ✓ **VGG16** - Deep CNN with 16 layers
- ✓ **ResNet50** - 50-layer network with residual connections
- ✓ **MobileNet** - Lightweight model for mobile devices
- ✓ **InceptionV3** - Multi-scale processing architecture
- ✓ **EfficientNetB0** - Optimized for accuracy and efficiency

Traditional ML

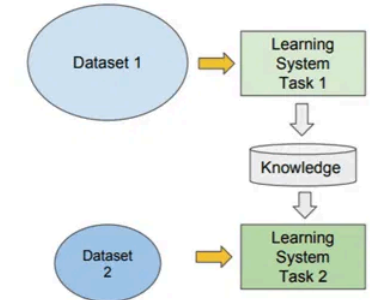
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

Transfer Learning

- Learning of a new task relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



Model Evaluation

Evaluation Metrics

- ✓ **Accuracy:** Overall correctness of predictions
- ✓ **Precision:** Ratio of correct positive predictions
- ✓ **Recall:** Ability to find all positive samples
- ✓ **F1-Score:** Harmonic mean of precision and recall

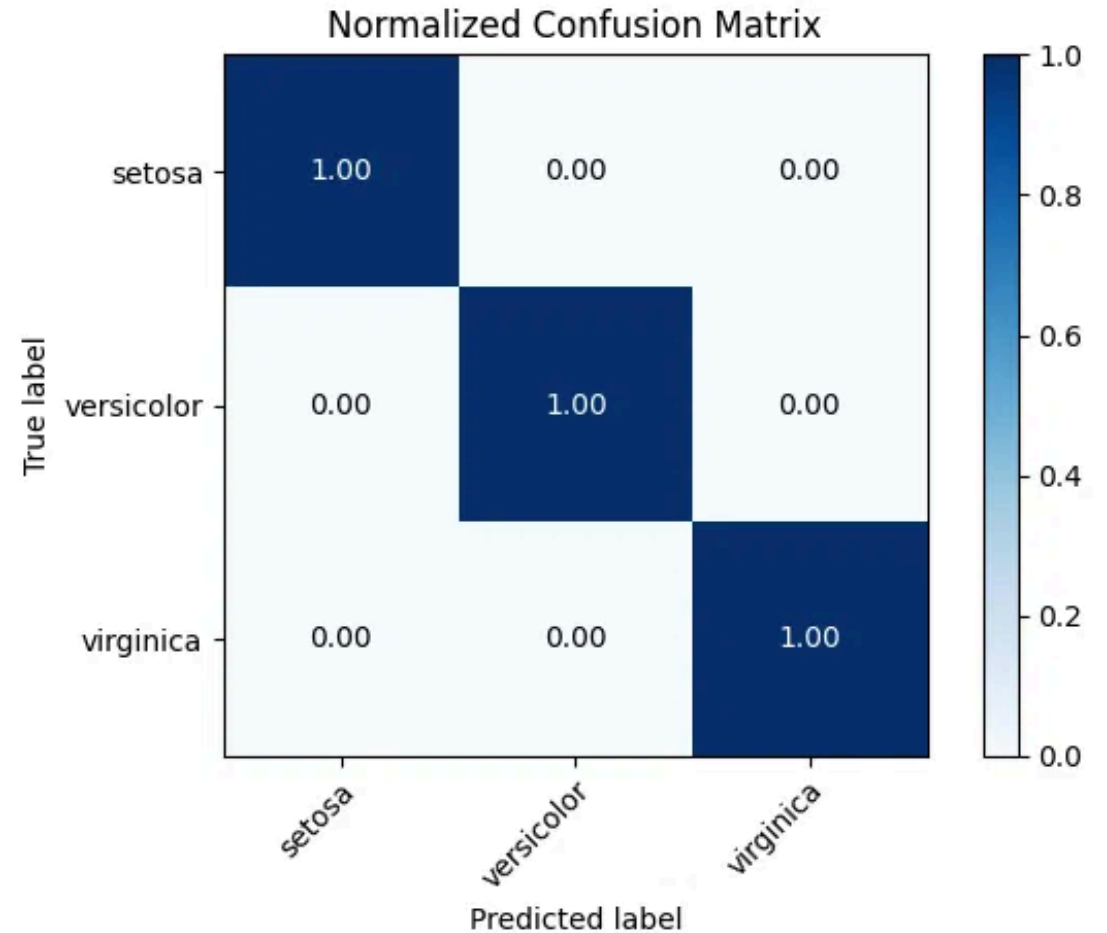
Model Comparison

We compare performance across multiple models:

- Custom CNN (baseline)
- VGG16, ResNet50, MobileNet
- InceptionV3, EfficientNetB0

Visualization Tools

Performance is visualized using **confusion matrices** and **training history plots** to identify strengths and weaknesses of each model.



Streamlit Deployment

What is Streamlit?

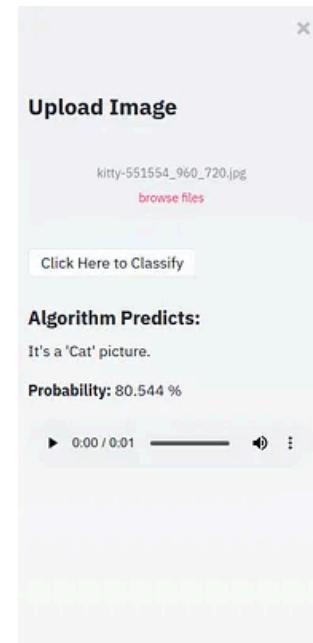
Streamlit is an open-source Python library that makes it easy to create and share custom web apps for machine learning and data science.

Key Features of Our Fish Classification App

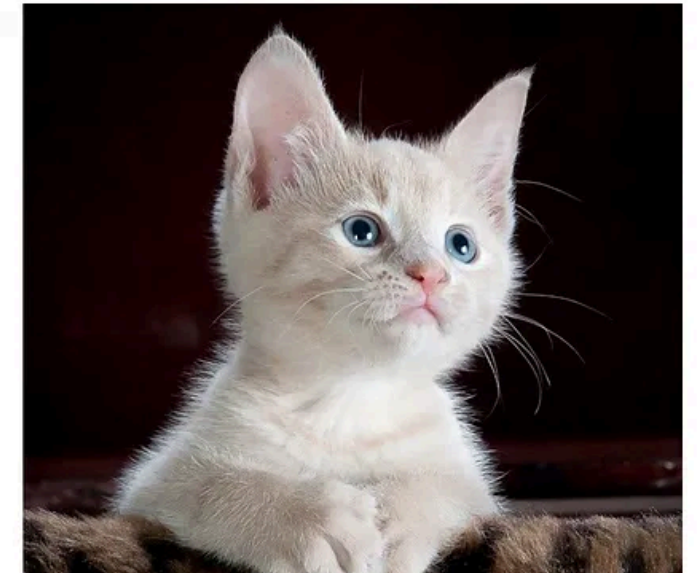
- 📁 User-friendly image upload interface
- 🐟 Real-time fish species prediction
- 📊 Confidence scores visualization
- 📱 Responsive design for various devices
- 🔗 Simple Python code implementation

Benefits

Allows non-technical users to easily interact with our trained model without coding knowledge. Perfect for field researchers and educational purposes.



Cat Image Classification App



Deployment Process

Step-by-Step Deployment

1 Save the Best Model

Export the highest accuracy model to .h5 format

```
model.save('fish_classifier.h5')
```

2 Create Streamlit App

Build a Python script with Streamlit components

```
streamlit run app.py
```

3 Set Up GitHub Repository

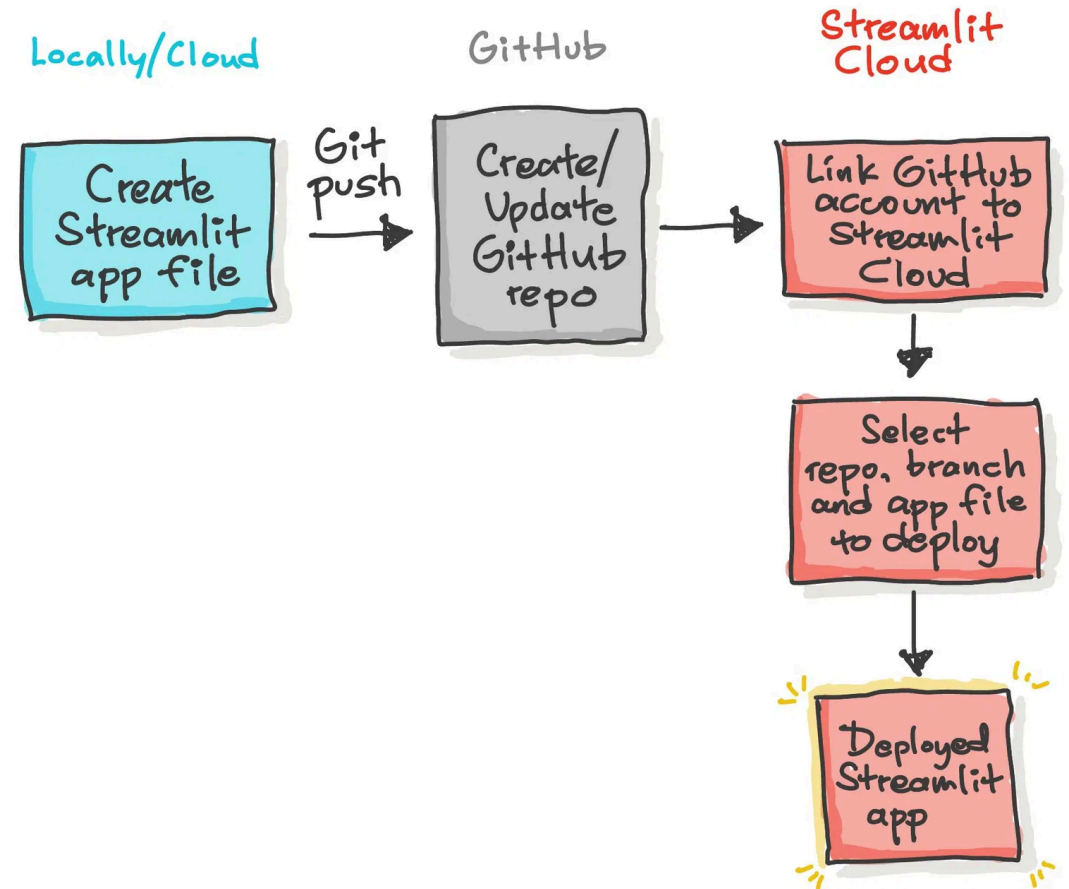
Create repo with model, app code, and requirements

4 Deploy to Streamlit Cloud

Connect GitHub repo to Streamlit Cloud for public access

5 Test & Share

Verify functionality and share the app URL



Project Deliverables



Trained Models

Custom CNN and fine-tuned pre-trained models (VGG16, ResNet50, MobileNet, InceptionV3, EfficientNetB0) saved in .h5 format



Streamlit Application

Interactive web app for real-time fish classification with user-friendly interface for image upload and prediction display



Python Scripts

Well-documented code for data preprocessing, model training, evaluation, and deployment



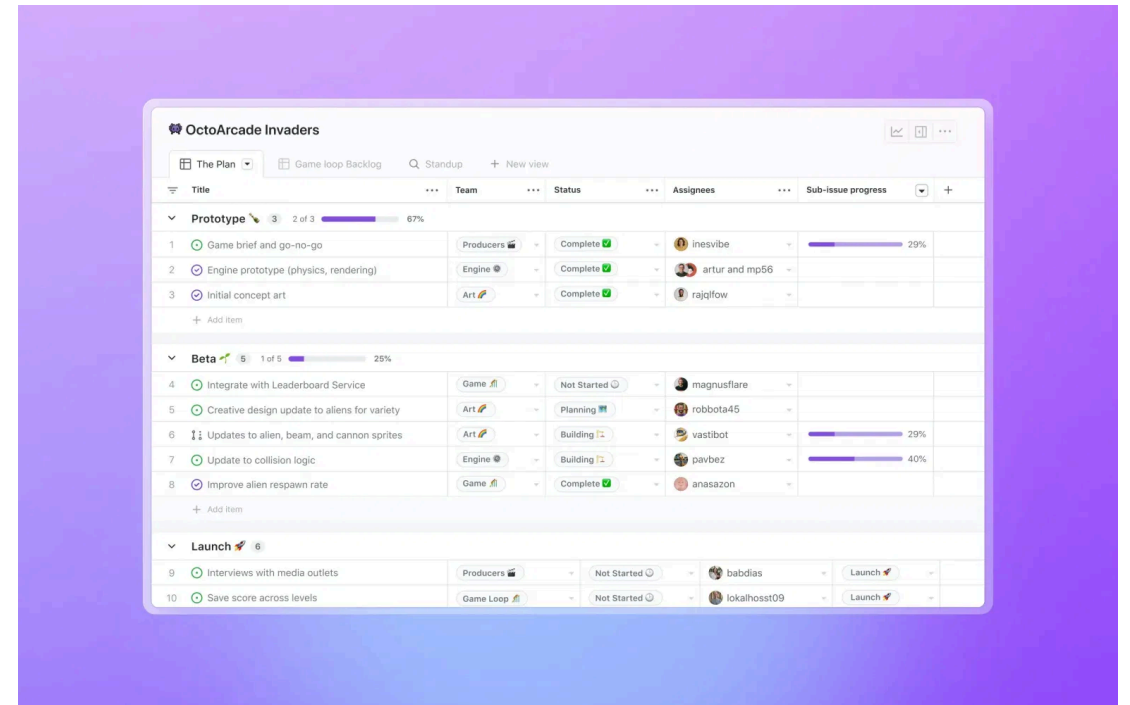
Comparison Report

Detailed analysis of model performance metrics and visualizations to identify the best model for fish classification



GitHub Repository

Complete codebase with comprehensive README documentation for easy reproduction and future development



Conclusion & Skills Gained

Project Summary

We successfully built and compared multiple deep learning models for fish species classification, deployed a user-friendly Streamlit application, and documented the entire process.

Key Skills Gained

 Deep Learning Python

TensorFlow/Keras

 Streamlit

Data Preprocessing

⇒ Transfer Learning

Model Evaluation

Visualization

Future Improvements

Expand the dataset with more species, implement ensemble methods for better accuracy, and optimize the model for mobile deployment.

